

Contents

Specification Review Report	1
1. Executive Summary	1
2. Overall Maturity	2
3. Findings Summary	2
4. Detailed Findings	2
F-001 — CLI normalization contract is incomplete	2
F-002 — Zero-rate inverse and payment-count rounding are undefined	2
F-003 — Rate solver algorithm is not reproducibly specified	3
F-004 — Decimal JSON serialization is unspecified	3
F-005 — Amortization final-row semantics conflict with row arithmetic	3
F-006 — Safe schedule bound has no value	4
F-007 — Error return vs. exception behavior is ambiguous	4
F-008 — CLI option combinations and exit mapping are incomplete	4
F-009 — Interpretation contract does not pin extracted-field provenance	4
F-010 — Real adapter failure and retry behavior is underspecified	5
F-011 — Observability/provenance fields are missing	5
F-012 — Known-value expected amount is not pinned	5
F-013 — Property-based inverse testing is not an acceptance obligation	5
5. Requirements Review	6
6. Interface and Data-Contract Review	6
7. State and Failure Review	6
8. Determinism and Algorithm Review	6
9. Edge-Case Review	6
10. Non-Functional Requirement Review	6
11. Security and Trust-Boundary Review	6
12. Observability and Provenance Review	6
13. Testing and Verification Review	7
14. Metrics and Evaluation Review	7
15. Traceability Review	7
16. Internal-Consistency Review	7
17. Architecture Review	7
18. Implementation-Agent Readiness	7
19. Quality Scorecard	7
20. Remediation Plan	8
P0 — Blocking	8
P1 — Important	8
P2 — Improvement	8
21. Final Verdict	8

Specification Review Report

1. Executive Summary

SPEC.md describes a coherent hybrid mortgage calculator with a strong deterministic/probabilistic boundary, explicit contracts, a useful error taxonomy, and an unusually complete initial acceptance suite. The four-pass review found several material ambiguities that could produce incompatible implementations, especially around normalized CLI inputs, zero-interest inverse calculations, rate-solver behavior, amortization precision, JSON serialization, and adapter failure semantics.

Findings: **3 HIGH, 8 MEDIUM, 2 LOW**. The most important corrections are incorporated into SPEC.md v0.2: a canonical normalization contract, explicit zero-rate and payment-count policies, a fully specified bisection solver, JSON decimal encoding, schedule final-row semantics, a concrete schedule bound, consistent

CLI exits, and adapter/tool failure behavior.

2. Overall Maturity

Pre-remediation: Level 2 — Implementable, but material semantic questions remained.

Post-remediation target: Level 3 — Implementation-grade. The updated specification is intended to let a competent coding agent implement the system with minimal semantic inference and let a verifier test conformance objectively.

3. Findings Summary

Severity	Count	IDs
HIGH	3	F-001, F-002, F-003
MEDIUM	8	F-004, F-005, F-006, F-007, F-008, F-009, F-010, F-011
LOW	2	F-012, F-013

4. Detailed Findings

F-001 — CLI normalization contract is incomplete

Severity: HIGH **Location:** §2 R-02/R-06, §4 C-01, §5.1

Observation The canonical request requires `periodic_rate` and `payments`, while the CLI example supplies annual `--rate` and `--term-years`. The spec does not define whether `--rate` is a percentage or decimal, whether `--term-years` may be combined with `--payments`, or which duplicate/conflicting flags win.

Why it matters Two implementations can interpret 6.5 as 6.5 or 0.065, and can choose different precedence for conflicting duration flags.

Potential consequence Equivalent CLI requests may produce different results or violate I-010.

Recommended resolution Pin CLI units and normalization: `--rate 6.5 --rate-period annual` means 6.5 percent; monthly rate input is decimal; `--term-years` maps to `payments = years * 12`; conflicting aliases are usage errors.

F-002 — Zero-rate inverse and payment-count rounding are undefined

Severity: HIGH **Location:** §2 R-03, §4 C-04, §6 I-003, §9 T-03/T-05

Observation The payment formula divides by `r`, and the payment-count logarithmic formula also becomes undefined at `r=0`, although E-03 says zero rate is valid. `calculate_payments` returns `int`, but no exact-integer/rounding policy is defined.

Why it matters The zero-interest path and the term inversion can produce incompatible behavior between implementations.

Potential consequence Valid zero-rate loans can fail, or a fractional computed term can be silently rounded up/down.

Recommended resolution Define zero-rate formulas explicitly: $M=P/n$, $P=M*n$, $n=P/M$ only when the quotient is within a configured integer tolerance; otherwise return `INVALID_PAYMENTS` or a dedicated non-integral-term error. Define term inversion for positive rates as exact-real computation followed by an explicit integer policy.

F-003 — Rate solver algorithm is not reproducibly specified

Severity: HIGH **Location:** §2 R-08, §4 C-04, §5.3, §6 I-006, §7 K-04

Observation The source allows bisection/Newton/Brent, but the spec later pins only an interval, tolerance, and iteration count. It does not define the objective function, endpoint/bracketing checks, midpoint stopping condition, or returned rate precision.

Why it matters Different solvers and stopping rules can produce different results and different convergence failures.

Potential consequence T-04 and T-09 cannot objectively distinguish conforming from non-conforming implementations.

Recommended resolution Pin bisection for v0.1: solve `payment(P,r,n)-M=0` over $[0,1]$, require a sign change or exact endpoint, stop when absolute residual or interval width is within tolerance, and return `SOLVER_CONVERGENCE` otherwise.

F-004 — Decimal JSON serialization is unspecified

Severity: MEDIUM **Location:** §4 C-02/C-05/C-09, §7 K-05

Observation Contracts use `Decimal`, while JSON has no `Decimal` type. The spec does not say whether currency/rates are serialized as JSON numbers or strings, nor how non-finite values are represented.

Why it matters Consumers and tests may parse the same result differently and lose precision.

Potential consequence The tool and CLI can disagree despite sharing a calculation result.

Recommended resolution Serialize canonical `Decimal` values as base-10 strings in JSON; serialize integer counts as JSON integers; prohibit NaN/Infinity; text mode applies presentation rounding.

F-005 — Amortization final-row semantics conflict with row arithmetic

Severity: MEDIUM **Location:** §4 C-06, §6 I-008, §8 E-12, §9 T-12/T-13

Observation The spec requires every row to satisfy `payment = principal + interest`, but permits the final row's principal component to be adjusted without specifying whether payment or interest may also change.

Why it matters Implementations can produce different final payments and fail either the row identity or final-balance assertion.

Potential consequence Schedule totals and displayed final payments diverge.

Recommended resolution Define each row from unrounded values; on the final row set `principal = prior_balance + interest`, `payment = principal + interest`, and `balance = 0`. Mark the final row as an adjusted payoff row if it differs from the regular payment.

F-006 — Safe schedule bound has no value

Severity: MEDIUM **Location:** §7 K-10, §8 E-04

Observation K-10 requires a documented safe bound but does not specify one.

Why it matters A bound is externally observable and affects which valid requests are accepted.

Potential consequence Implementations choose incompatible resource limits.

Recommended resolution Pin `max_schedule_payments = 1200` for v0.1; requests above it return `INVALID_PAYMENTS` before allocation.

F-007 — Error return vs. exception behavior is ambiguous

Severity: MEDIUM **Location:** §4 C-03/C-04/C-05, §5.1

Observation The spec says the calculation layer “MUST return or raise” typed errors, while the tool requires an error envelope and CLI exits depend on error categories.

Why it matters Callers and tests cannot know whether to catch exceptions or inspect a result union.

Potential consequence The CLI, tool, and service can expose different failure semantics.

Recommended resolution Make the public service and tool return a discriminated result envelope; internal calculator functions MAY raise a private typed exception, which the service MUST convert. Define one exit mapping for every error code.

F-008 — CLI option combinations and exit mapping are incomplete

Severity: MEDIUM **Location:** §5.1

Observation `--rate`, `--term-years`, `--payments`, `--rate-period`, and `--include-schedule` combinations are not fully constrained. `calculate` lists an unsupported-scope exit despite having no scope input, and tool failures are mapped inconsistently between commands.

Why it matters CLI behavior is part of the primary interface.

Potential consequence Valid-looking invocations can be interpreted differently, and shell automation cannot rely on exits.

Recommended resolution Define aliases, mutual exclusions, required combinations, and a single exit table: 0 success; 2 usage/validation/clarification; 3 solver/tool; 4 unsupported scope; 5 model failure.

F-009 — Interpretation contract does not pin extracted-field provenance

Severity: MEDIUM **Location:** §4 C-07/C-08, §6 I-010, §9 T-20..T-26

Observation `Interpretation` contains only a request, clarification, and free-form assumptions. It does not identify whether values were explicit or derived (for example, purchase price minus down payment), or preserve source units.

Why it matters A verifier cannot distinguish a correctly derived principal from an invented one.

Potential consequence Natural-language evaluation can pass while hiding unsafe assumptions.

Recommended resolution Add structured `FieldEvidence` with field name, source text, normalized value, and derivation; require every populated primary field to have evidence and every derived value to have an explicit assumption.

F-010 — Real adapter failure and retry behavior is underspecified

Severity: MEDIUM **Location:** §2 R-18, §4 C-07, §7 K-09, §8 E-11

Observation The real adapter is optional, but timeout, malformed model output, explanation failure, and calculator-tool failure are not assigned precise retry/terminal behavior.

Why it matters The adapter boundary is a stated reliability boundary.

Potential consequence One implementation retries indefinitely while another silently falls back to model arithmetic.

Recommended resolution Pin one bounded attempt for `interpret` and `explain` in v0.1; map time-out/malformed output to `MODEL_ERROR`, tool failures to the tool’s original error, and prohibit arithmetic fallback. The mock path remains the only automated path.

F-011 — Observability/provenance fields are missing

Severity: MEDIUM **Location:** §4 C-02/C-05/C-07, §9

Observation Results do not record schema version, solver configuration, adapter identity, or assumptions in a machine-readable way.

Why it matters An engineer cannot reliably reproduce why two outputs differ.

Potential consequence Regression tests and user reports lose calculation provenance.

Recommended resolution Add `schema_version`, `calculation_config`, `adapter`, and `assumptions` to the top-level response envelope; include them in JSON and test their presence.

F-012 — Known-value expected amount is not pinned

Severity: LOW **Location:** §9 T-01

Observation T-01 says “independently verified” but does not state the expected payment.

Why it matters A test author must independently choose a reference value.

Potential consequence Different precision/rounding references produce inconsistent tests.

Recommended resolution Pin the reference: for \$100,000, 6% annual, 360 payments, expected payment is \$599.550525152752; text output rounds to \$599.55.

F-013 — Property-based inverse testing is not an acceptance obligation

Severity: LOW **Location:** source appendix §A.16, §9

Observation The source calls for property-based testing, but the spec only makes it optional in dependencies and does not require a generated inverse test.

Why it matters A central educational objective is not objectively assessed.

Potential consequence The implementation may pass fixed examples while inverse relationships fail elsewhere.

Recommended resolution Add T-33 requiring generated valid-loan round trips for payment/principal and payment/rate within defined tolerances, with deterministic seed and bounded sample count.

5. Requirements Review

The requirements are generally observable and use normative language correctly. R-02, R-06, and R-08 required additional normalization and algorithm detail. R-18 required an explicit mapping from failure classes to public outcomes. No contradictory high-level product goal was found.

6. Interface and Data-Contract Review

The contracts establish useful module boundaries, but JSON serialization, service-result error handling, field provenance, and configuration metadata were underspecified. These are resolved in v0.2 by making envelopes discriminated and Decimal serialization canonical.

7. State and Failure Review

The state model is appropriate for a stateless request. It lacked explicit adapter retry limits and a precise final amortization-row rule. v0.2 makes model attempts bounded and defines the adjusted payoff row.

8. Determinism and Algorithm Review

The formulas are directionally correct. The main determinism risks were zero-rate paths, integer payment-count policy, and the rate solver's unspecified bisection mechanics. These are blocking for reproducible verification and are addressed in v0.2.

9. Edge-Case Review

The spec covers many financial and scope boundaries. The safe schedule bound and non-integral zero-rate term needed concrete outcomes. CLI conflicting aliases also needed an edge case.

10. Non-Functional Requirement Review

The $O(n)$ amortization constraint and offline requirement are measurable. The v0.2 bound of 1,200 schedule rows makes the resource constraint executable. No latency requirement is necessary for this synchronous MVP.

11. Security and Trust-Boundary Review

The deterministic/probabilistic boundary is strong. v0.2 further forbids arithmetic fallback after model/tool failure and requires provenance for derived natural-language fields. Secrets and provider configuration remain outside the deterministic core.

12. Observability and Provenance Review

The initial spec had a disclaimer but no machine-readable provenance. v0.2 adds adapter, assumptions, schema version, and solver configuration to output envelopes.

13. Testing and Verification Review

The initial suite covered most formulas and boundaries. v0.2 pins the known-value oracle, adds JSON serialization and exit tests, and makes property-based inverse testing an acceptance obligation.

14. Metrics and Evaluation Review

The primary outputs are financial quantities rather than operational metrics. Precision/tolerance and rounding were clarified; no additional performance metric is required for this scope.

15. Traceability Review

The initial matrix covered all identifiers but did not include the low-level serialization, provenance, and property-based obligations. v0.2 extends the matrix with F-001..F-013 remediation links and T-33.

16. Internal-Consistency Review

The principal inconsistency was the coexistence of canonical monthly fields with annual-rate/years CLI examples without a formal mapping. Error exits and amortization final-row arithmetic also needed reconciliation. These are resolved in v0.2.

17. Architecture Review

The proposed dependency direction is sound: CLI/service -> tool/calculator; LLM adapter -> tool; deterministic core does not import the model. No redesign is required.

18. Implementation-Agent Readiness

Before remediation: NO — MATERIAL QUESTIONS REMAIN.

After remediation: YES. The remaining choices are implementation freedom, not conformance-changing semantic questions.

19. Quality Scorecard

Dimension	Score
Scope clarity	5
Terminology	4
Requirement precision	4
Interface completeness	3
Data-contract completeness	3
State/lifecycle definition	4
Algorithm precision	3
Failure semantics	3
Edge-case coverage	4
Non-functional requirements	4
Security specification	4
Observability/provenance	2
Testability	4
Evaluation/metrics	3
Traceability	4
Internal consistency	3
Architecture consistency	5
Implementation readiness	3

Pre-remediation total: 65/90. The score is not a maturity average; it highlights the specific dimensions requiring remediation.

20. Remediation Plan

P0 — Blocking

- F-001: Pin CLI normalization and conflicting-option behavior.
- F-002: Pin zero-rate formulas and payment-count integer policy.
- F-003: Pin the v0.1 bisection solver.

P1 — Important

- F-004: Define Decimal JSON encoding.
- F-005: Define final amortization-row adjustment.
- F-006: Pin the maximum schedule length.
- F-007: Define result-envelope error handling.
- F-008: Reconcile CLI option combinations and exit codes.
- F-009: Add interpretation field evidence.
- F-010: Bound real-adapter attempts and forbid arithmetic fallback.
- F-011: Add machine-readable provenance.

P2 — Improvement

- F-012: Pin the known-value oracle.
- F-013: Require property-based inverse testing.

All findings were applied to `SPEC.md`; the report is retained as the review record.

21. Final Verdict

Specification maturity: **Level 3 — Implementation-grade after remediation**

Implementation readiness: **READY**

Primary blocker: **NONE after the v0.2 remediation integrated into SPEC.md.**

Most important improvement: **The deterministic normalization, solver, precision, and error contracts now make equivalent implementations and verification outcomes materially consistent.**